ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) | 979-8-3315-6701-9/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICASSP55912.2026.11463591

EVOALPHA: AN LLM-ENHANCED EVOLUTIONARY FRAMEWORK FOR FORMULAIC ALPHA MINING

Yihong Wu[1] *Chengcan Lou*[1] *Jiayuan Zhang*[2] *Sirong Chen* [3] *Yan Yang*[1*]

[1]School of Computer Science and Technology, East China Normal University, China
[2]College of Design and Engineering, National University of Singapore, Singapore
[3]School of Computing and Communications, Lancaster University, UK

ABSTRACT

Formulaic alpha mining aims to discover mathematical formulas from raw stock data to predict market trends. Existing methods, based on expression tree search, suffer from large search spaces, meaningless results, and inefficient evaluation. We propose EvoAlpha, an LLM-guided framework that generates high-quality alphas with code and explanations. It leverages an LLM-enhanced Evolutionary Chain-of-Thought for analysis, crossover, and mutation. Then it uses a Code Evaluator to convert alphas into executable code, and applies a diversity checker to ensure variety. Experiments on CSI300 and CSI500 show EvoAlpha achieves state-of-the-art performance, improves cumulative returns by at least 12% in backtesting, significantly enhances the quality and efficiency of alpha mining, and can generalize to other feature engineering tasks.

***Index Terms*—** Large Language Models, Evolutionary Algorithms, Trading Signal Prediction, Stock Trend Prediction

1. INTRODUCTION

In quantitative investing, alphas are feature patterns modeling market data (e.g., open and close prices) to predict returns and inform trading signals. They are categorized into black-box and formulaic alphas. Black-box alphas, powered by machine-learning models [1, 2, 3, 4, 5], capture complex patterns but are opaque and prone to overfitting. Formulaic alphas, represented by mathematical formulas, provide financial interpretations and are easier to analyze. For example, "$(close - open)/((high - low) + 0.01)$" is an alpha that represents the magnitude of price movement. This alpha generates scores, with higher values indicating higher expected returns. Despite their advantages, constructing alphas demands expertise and manual effort, underscoring the importance of automated methods.

Existing methods for alpha mining mainly focus on evolutionary algorithms [6, 7]. These frameworks employ evolutionary operators to evolve a population of alphas, refining them over successive generations. In order to balance the search space and the quality of the generated alphas, reinforcement learning is introduced, which models the alpha mining as a Markov decision-making process [8, 9, 10]. However, the construction of formulaic alphas involves enormous operators and operands, which results in a large search space and makes the exploration inefficient. Moreover, these methods ignore financial semantics behind formulaic alphas, potentially leading to the generation of numerous meaningless alphas. For example, "$EMA(Add(Constant(-0.01), Constant(-30.0)), 30)$" is a typical example generated by them. Such a formula is merely a random combination of operators and operands, completely lacking financial insights. Furthermore, these methods depend on external parsers to convert the generated alphas into expression trees, which further restricts their flexibility in both application and evaluation.

Recent advancements in Large Language Models (LLMs) show promising performance in various tasks [11, 12], providing opportunities for alpha mining [13, 14, 15]. With extensive background knowledge, LLMs can anticipate and prune incorrect paths when exploring, narrowing the exploration spaces to enhance the evolutionary process [16, 17, 18, 19, 20, 21]. Moreover, LLMs exhibit strong comprehension and reasoning abilities [22], enabling them to understand and analyze the financial semantics behind alphas. This can improve the quality of the generated alphas and make them more reasonable. Furthermore, LLMs' proficiency in code generation [23] enables the conversion of alphas into executable code.

In this paper, we propose a novel formulaic alpha mining framework called EvoAlpha, which leverages LLMs to guide the evolutionary process, reduce the search space, and generate diverse, high-quality alphas. Specifically, we introduce an LLM-enhanced Evolutionary Chain-of-Thought (CoT) [22], which facilitates analysis, crossover, and mutation step-by-step to generate reasonable alphas from selected parents. Next, we present a Code Evaluator that translates these alphas into executable code, eliminating the need for external parsers and improving evaluation efficiency. Finally, we design a diversity checker to filter and select more diverse alphas, ensuring the population is updated effectively. We conduct experiments using real data from the CSI300 and CSI500 indices. The results demonstrate that our method outperforms strong baselines, achieving at least a 12% improvement in cumulative returns during backtests. Our contributions are summarized as follows:

1. EvoAlpha is the first automated framework for mining formulaic alphas using LLM-enhanced evolutionary algorithms (EAs).
2. We introduce Evolutionary CoT, which leverages the knowledge and powerful capabilities of LLMs to reduce the search space and generate more financially reasonable alphas.
3. We propose the Code Evaluator as a bridge between LLMs and formulaic alpha mining, translating alphas into executable code to simplify evaluation and application.
4. Experimental results show that EvoAlpha achieves state-of-the-art (SOTA) performance, enhancing quality and efficiency while generalizing to other feature engineering tasks. Our code is available at https://github.com/wuyihong994/EvoAlpha/.

2. METHODOLOGY

2.1. Problem Formulation

A formulaic alpha is defined as a mathematical formula $f(\cdot)$. On each trading day $t \in 1, 2, ..., T$, it transforms the m features of the

*Corresponding Author. E-mail: yanyang@cs.ecnu.edu.cn

Authorized licensed use limited to: Indian Institute of Technology - Jodhpur. Downloaded on October 18,2026 at 14:43:35 UTC from IEEE Xplore. Restrictions apply.

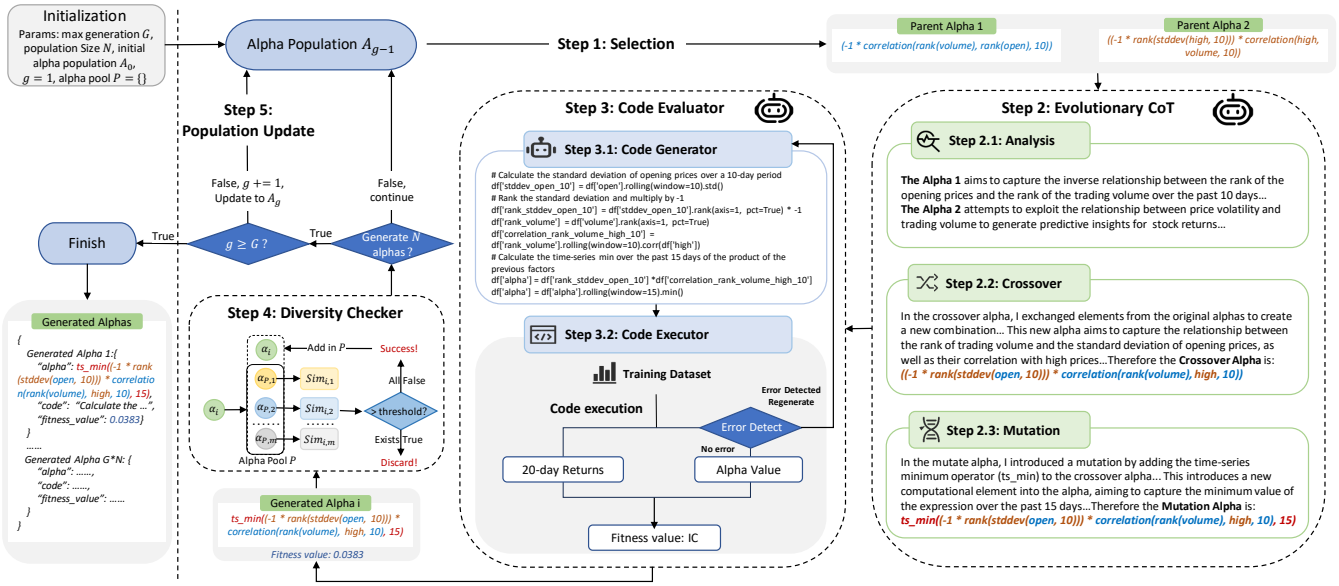


Fig. 1. The framework of EvoAlpha. It first selects two parent alphas from the population and then generates a new alpha through Evolutionary CoT, which subsequently is evaluated by the Code Evaluator. Then, it will be filtered by the diversity checker. Each generation g will generate N alphas. Then, these generated alphas are merged with the original population, and the top N alphas are retained as the population for the next generation. This process iterates until g reaches the predefined G.

i-th stock $X_{t,i} \in \mathcal{R}^m$ (e.g., open and close prices) into an alpha value $\alpha_{t,i} = f(X_{t,i}) \in \mathcal{R}^1$. Thus, $\alpha_t = (\alpha_{t,1}, ..., \alpha_{t,n})$ represents the alpha values for all n stocks at date t. To measure alpha effectiveness, we use the Information Coefficient (IC), defined as the Pearson correlation between alpha values and future stock returns:

$$IC = \frac{1}{T}\sum_{t=1}^{T} corr(\alpha_t, r_t^h) \tag{1}$$

where $r_t^h = (r_{t,1}^h, ..., r_{t,n}^h)$ denotes stock returns at date t over a holding period h, with

$$r_{t,i}^h = \frac{close_{t+h,i} - close_{t,i}}{close_{t,i}} \tag{2}$$

Similarity between alphas is measured by the Pearson correlation of their values, called Mutual Information Coefficient (mutIC):

$$mutIC = \frac{1}{T}\sum_{t=1}^{T} corr(\alpha_{1,t}, \alpha_{2,t}) \tag{3}$$

IC quantifies an alpha's predictive power; higher IC indicates stronger relevance to future returns. Negative IC can be flipped by multiplying by -1, so IC is generally assumed non-negative. The goal of alpha mining is to discover alphas with high predictive capacity.

2.2. EvoAlpha

In this section, we provide an overview of EvoAlpha (Figure 1), a framework that leverages an LLM-enhanced EA to discover formulaic alphas. To implement this, we employ one of the most widely used EAs, the genetic algorithm [24], which has well-defined evolutionary operators and is better suited for formulaic alpha mining compared to other EAs. Our framework is initialized with Alpha101 [25] and consists of five components. When initialization, Alpha101 are evaluated by IC, which serves as our fitness function, and the top N alphas from the initial population A_0. Then EvoAlpha iteratively generates alphas through the five components, starting from $g = 1$. First, the Selection component employs the roulette wheel selection to choose parent alphas from A_{g-1}. Then, the Evolutionary CoT leverages LLMs to generate offspring alphas through three steps: analysis, crossover, and mutation. Next, the converts them into executable code, followed by the Diversity Checker. At the end of each generation g, the population is updated to A_g, and this process continues until g reaches a predefined number G.

2.2.1. *Selection*

In each iteration, EvoAlpha first applies roulette wheel selection [26] to choose two parent alphas from the population. For a population A_{g-1} with N alphas, each alpha i has a fitness value s_i, and its probability of being selected as a parent is $p_i = s_i / \sum_{j=0}^{N} s_j$. This reflects survival of the fittest, where higher-fitness alphas are more likely to reproduce, improving the fitness of the next generation A_g.

2.2.2. *Evolutionary CoT*

To enhance evolution, we propose Evolutionary CoT, which uses LLMs' knowledge and reasoning capabilities for evolution. Based on two selected parent alphas, it guides an LLM to generate an offspring alpha a' via three steps: analysis, crossover, and mutation.

Analysis: Mimicking human experts, we input two parent alphas into a prompt template and utilize an LLM to analyze them. As shown in Figure 1, the output explains their structure and financial meaning, improving both user understanding and the LLM's comprehension, which enhances subsequent crossover and mutation.

Crossover: Using parent alphas' analysis, the LLM selects and recombines their components to generate a crossover alpha, providing explanations and financial analysis to ensure reasonableness. As

Authorized licensed use limited to: Indian Institute of Technology - Jodhpur. Downloaded on October 18,2026 at 14:43:35 UTC from IEEE Xplore. Restrictions apply.

Method	CSI300				CSI500			
	20 days		30 days		20 days		30 days	
	IC	ICIR	IC	ICIR	IC	ICIR	IC	ICIR
MLP	0.0337	0.2882	0.0370	0.2984	0.0253	0.2350	0.0279	0.2683
	(0.0130)	(0.1139)	(0.0127)	(0.0952)	(0.0067)	(0.0507)	(0.0033)	(0.0242)
XGBoost	0.0506	**0.4129**	0.0582	0.4647	0.0334	0.2852	0.0437	0.3820
	(0.0009)	(0.0084)	(0.0011)	(0.0077)	(0.0011)	(0.0090)	(0.0005)	(0.0057)
CatBoost	0.0379	0.3140	0.0421	0.3498	0.0228	0.1812	0.0375	0.2991
	(0.0006)	(0.0047)	(0.0018)	(0.0160)	(0.0022)	(0.0173)	(0.0002)	(0.0023)
LightGBM	0.0490	0.4078	0.0516	0.4268	0.0269	0.2255	0.0328	0.2991
	(0.0002)	(0.0012)	(0.0004)	(0.0048)	(0.0009)	(0.0084)	(0.0015)	(0.0059)
Alpha101	0.0114	0.1197	0.0210	0.2137	0.0080	0.0998	0.0093	0.1174
gplearn	0.0490	0.3258	0.0584	0.4341	0.0435	0.3124	0.0551	0.3947
	(0.0070)	(0.0074)	(0.0034)	(0.0182)	(0.0005)	(0.0140)	(0.0010)	(0.0048)
AlphaGen	0.0532	0.3295	0.0641	0.3916	0.0436	0.3404	0.0527	0.4078
	(0.0002)	(0.0178)	(0.0015)	(0.0123)	(0.0052)	(0.0216)	(0.0058)	(0.0200)
EvoAlpha	**0.0618**	0.3581	**0.0821**	**0.4765**	**0.0504**	**0.3472**	**0.0651**	**0.4569**
	(0.0010)	(0.0053)	(0.0008)	(0.0079)	(0.0025)	(0.0195)	(0.0023)	(0.0159)

Table 1. Main Results on CSI300 and CSI500. All experiments are run 3 times to ensure robustness. Values outside the parenthesis are the mean, and values inside the parenthesis are the standard deviation.

shown in Figure 1, parent analysis help identify valuable components, enabling the new alpha to inherit strengths from both parents.

Mutation: Next, we mutate the crossover alpha by replacing, adding, or deleting elements, generating a new alpha with an explanation. As shown in Figure 1, the mutation can introduce functions like a 15-period minimum to highlight lower values. Mutation maintains population diversity, prevents premature convergence, and explores new regions to discover more effective alphas.

Unlike previous methods, our approach leverages LLMs' external knowledge to narrow the extensive search space, as it can identify and prune incorrect search paths. As a result, EvoAlpha can generate more meaningful and high-quality alphas.

2.2.3. Code Evaluator

To overcome the limitations of previous expression-format alphas, we propose a more flexible approach by using Code Evaluator. This module generates executable code for alphas, enabling direct application to stock data and code execution to calculate fitness values.

Code Generator: We sample data from the training dataset and provide these examples along with task details as input to the LLM. This guides the LLM in generating code to implement the alpha.

Code Executor: This module applies the generated code to stock data from the training set, calculating the similarity between the alpha values and stock returns. It runs the code to verify executability. If any issues arise during execution, the errors are recorded and fed back to the LLM for regeneration. Additionally, the executor masks future information during each trading date to prevent lookahead, maintaining the correctness of the evaluation.

By utilizing code as an interface for formulaic alphas, we expand its scalability, making it easier to represent complex logical relationships in a clearer manner. Furthermore, code enables more efficient adaptation and allows alphas to be directly executed, making future modifications and applications more convenient.

2.2.4. Diversity Checker

To generate more diverse alphas, we propose a diversity checker. We maintain an alpha pool P containing all generated alphas. Each new alpha α' is compared with those in P using mutICs to measure similarity. If any mutIC exceeds 0.9, α' is discarded; otherwise, it is added to P and the new population A'_g. This process ensures that the generated alphas maintain greater diversity.

2.2.5. Population Update

In our method, each generation g produces N alphas. After generating N valid alphas, EvoAlpha updates the population by sorting A'_g and A_{g-1} by fitness and selecting the top N alphas to form A_g. This ensures that only the most promising alphas are retained, improving overall population performance over generations.

3. EXPERIMENTS

3.1. Experiment Setting

We primarily use GPT-3.5-turbo [27] to perform the Evolutionary CoT and generate code for evaluation. We also explore different LLMs such as GPT-4o-mini [27], Deepseek-v2.5 [28], and LLaMA-3-70B-instruct [29]. We set the temperature to 1.0, the population size $N = 10$ and the number of generations $G = 50$.

Dataset: Our experiments are conducted on the Chinese A-shares stock market via baostock[1], focusing on CSI300 and CSI500 indices. The dataset, spanning 15 years of daily stock data, is split into training (2009/01/01-2018/12/31), validation (2019/01/01-2020/12/31), and testing (2021/01/01-2024/01/01) datasets. The stock data includes six key features: open, high, low, close, volume, and vwap. Returns are calculated using daily close prices, with two targets: 20-day and 30-day returns.

Baselines: We compare EvoAlpha with two types of baselines: black-box and formulaic alpha methods. Black-box methods include **MLP**, **XGBoost** [1], **LightGBM** [3], and **CatBoost** [2]. Their implementations and hyperparameters are set according to the open-source library Qlib [30]. Formulaic alpha methods include: **Alpha101** [25], a set of 101 commonly used formulaic alphas; **gplearn** [2], a symbolic regression library that employs genetic programming

[1]http://www.baostock.com

[2]https://github.com/trevorstephens/gplearn

Authorized licensed use limited to: Indian Institute of Technology - Jodhpur. Downloaded on October 18,2026 at 14:43:35 UTC from IEEE Xplore. Restrictions apply.

to evolve mathematical expressions; **AlphaGen** [8], a framework for formulaic alpha mining by reinforcement learning.

Evaluation Metrics: We use two industry-standard metrics: **IC**, which measures the correlation between alpha value and stock returns, indicating predictive power; **ICIR**, the ratio of IC to its standard deviation, reflecting predictive stability. For methods generating multiple alphas (Alpha101, gplearn, and ours), we sort alphas by IC and take the top 50 for averaging as the evaluation metric.

3.2. Main Results

Table 1 presents that EvoAlpha achieves the best IC and ICIR. This shows its effectiveness in discovering alphas highly correlated with actual returns while maintaining stability. MLP and CatBoost perform poorly due to weak handling of financial features. LightGBM and XGBoost perform better but still lag behind AlphaGen, reflecting the challenges of black-box methods. Alpha101 ranks lowest due to long-term public availability. Gplearn performs well by generating many alphas, naturally including effective ones. AlphaGen is competitive but still lags behind ours.

To analyze the quality of the generated alphas, we evaluate them on CSI300 with $h = 20$ and record the number of them. As shown in Table 2, gplearn generates 2,000 alphas but underperforms ours, which generates only 500 alphas. This shows EvoAlpha reduces the search space while producing fewer but higher-quality alphas. To assess diversity, we sample 100 pairs of alphas, compute their similarity, and repeat the process to report mean and standard deviation, denoted as **correlation**. Table 2 shows our method achieves lower correlation than gplearn, indicating higher diversity. As EvoAlpha integrates LLMs and a diversity checker to enhance diversity, we further analyze their contributions with an ablation study. "- w/o DC" represents EvoAlpha without the diversity checker. Results show that both the diversity checker and the LLM reduce correlation.

Method	IC	Correlation	Nums
gplearn	0.0490 ± 0.0070	0.5777 ± 0.1327	2234
EvoAlpha	**0.0618 ± 0.0010**	**0.1399 ± 0.0647**	**500**
- w/o DC	0.0597 ± 0.0008	0.1848 ± 0.0749	**500**

Table 2. The overall quality of the generated alphas.

We also compare different LLMs. Table 3 shows their performance, quality, and cost. From Table 3, GPT-3.5-turbo shows the best performance and lowest correlation. DeepSeek-v2.5 is slightly worse than GPT-4o-mini but surpasses most baselines.

Model Name	IC	Correlation	Cost
llama3-70b	0.0392 ± 0.0072	0.2464 ± 0.1294	-
deepseek-v2.5	0.0503 ± 0.0015	0.3126 ± 0.0972	0.68
gpt-4o-mini	0.0548 ± 0.0054	0.1844 ± 0.0585	1.5
gpt-3.5-turbo	**0.0618 ± 0.0010**	**0.1399 ± 0.0647**	10

Table 3. The quality of generated alphas and cost by different LLMs.

3.3. Backtest Results & Ablation Study

We conduct backtests on CSI300 to simulate real trading. Since EvoAlpha, gplearn, and Alpha101 involve multiple alphas, we train a regression model to combine the top 10 IC-ranked alphas. We use a top-k/drop-n strategy: sort stocks by alpha, select top k, sell held stocks below k, and evenly allocate investments. To cut trading costs, max n stocks are bought/sold daily; we set $k = 5$ and $n = 5$. Figure 2 shows EvoAlpha has the highest cumulative profit.

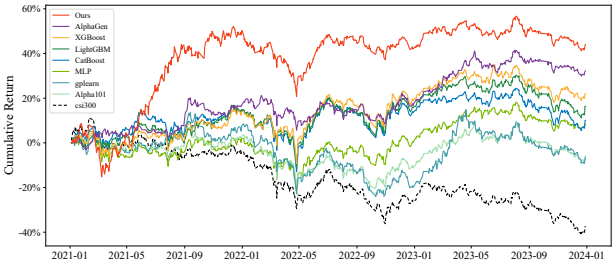
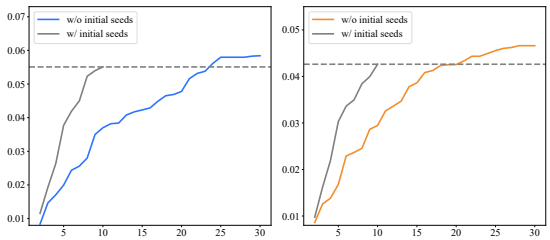


Fig. 2. Backtest result on CSI300.



(a) The Performance of IC on CSI300 (b) The Performance of IC on CSI500

Fig. 3. Ablation study of initial seeds.

We also investigate how Evolutionary CoT's three components affect its performance and Table 4 shows all components work.

CSI300	20 days IC	20 days ICIR	30 days IC	30 days ICIR
EvoAlpha	**0.0551**	**0.3294**	**0.0731**	**0.4349**
w/o analysis	0.0407	0.2941	0.0550	0.3927
w/o crossover	0.0344	0.2659	0.0474	0.3751
w/o mutation	0.0391	0.2819	0.0559	0.4125

Table 4. Ablation study of Evolutionary CoT in EvoAlpha.

We then investigate EvoAlpha's generalizability by examining if initial seeds are necessary. Previous experiments use Alpha101 as seeds, but in many applications, pre-existing seeds are unavailable. To address this, we implement a cold-start strategy, where an LLM generates 10 seeds as the initial population. We track the performance with holding days $h = 20$(Figure 3); with x-axis as generations and y-axis as average IC, "w/o initial seeds" achieves performance comparable to "w/ initial seeds". This proves EvoAlpha's generalizability, applicable to tasks even without high-quality seeds.

4. CONCLUSION

We proposes EvoAlpha, an automated framework using LLMs to enhance EAs for formulaic alpha mining. It has two key components: Evolutionary CoT, which leverages LLMs' knowledge and reasoning for evolution, performing analysis, crossover, mutation to generate offspring alphas; Code Evaluator, which translates alphas to executable code, streamlining evaluation and eliminating reliance on external parser. Experiments show EvoAlpha achieves new SOTA in metrics and backtests; we also demonstrate its generalizability to other feature engineering tasks without initial seeds.

Authorized licensed use limited to: Indian Institute of Technology - Jodhpur. Downloaded on October 18,2026 at 14:43:35 UTC from IEEE Xplore. Restrictions apply.

5. REFERENCES

[1] Tianqi Chen and Carlos Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, New York, NY, USA, 2016, KDD '16, p. 785–794, Association for Computing Machinery.

[2] Liudmila Prokhorenkova, Gleb Gusev, et al., "Catboost: unbiased boosting with categorical features," in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, Red Hook, NY, USA, 2018, NIPS'18, p. 6639–6649, Curran Associates Inc.

[3] Guolin Ke, Qi Meng, et al., "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds. 2017, vol. 30, Curran Associates, Inc.

[4] Qianggang Ding, Sifan Wu, Hao Sun, Jiadong Guo, and Jian Guo, "Hierarchical multi-scale gaussian transformer for stock movement prediction.," in *IJCAI*, 2020, pp. 4640–4646.

[5] Kelvin JL Koa, Yunshan Ma, et al., "Diffusion variational autoencoder for tackling stochasticity in multi-step regression stock price prediction," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, 2023, pp. 1087–1096.

[6] Tianping Zhang, Yuanqi Li, et al., "Autoalpha: an efficient hierarchical evolutionary algorithm for mining alpha factors in quantitative investment," 2020.

[7] Can Cui, Wei Wang, Meihui Zhang, Gang Chen, Zhaojing Luo, and Beng Chin Ooi, "Alphaevolve: A learning framework to discover novel alphas in quantitative investment," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, SIGMOD '21, p. 2208–2216.

[8] Shuo Yu, Hongyan Xue, Xiang Ao, Feiyang Pan, Jia He, Dandan Tu, and Qing He, "Generating synergistic formulaic alpha collections via reinforcement learning," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023.

[9] Tao Ren, Ruihan Zhou, Jinyang Jiang, Jiafeng Liang, Qinghao Wang, and Yijie Peng, "Riskminer: Discovering formulaic alphas via risk seeking monte carlo tree search," in *Proceedings of the 5th ACM International Conference on AI in Finance*, 2024, pp. 752–760.

[10] Feng Xu, Yan Yin, Xinyu Zhang, Tianyuan Liu, Shengyi Jiang, and Zongzhang Zhang, "Alpha^2: Discovering logical formulaic alphas using deep reinforcement learning," *arXiv preprint arXiv:2406.16505*, 2024.

[11] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[12] Karthik Valmeekam, Matthew Marquez, et al., "On the planning abilities of large language models-a critical investigation," *Advances in Neural Information Processing Systems*, vol. 36, pp. 75993–76005, 2023.

[13] Saizhuo Wang, Hang Yuan, et al., "Alpha-gpt: Human-ai interactive alpha mining for quantitative investment," *arXiv preprint arXiv:2308.00016*, 2023.

[14] Zhizhuo Kou, Holam Yu, Jingshu Peng, and Lei Chen, "Automate strategy finding with llm in quant investment," *arXiv preprint arXiv:2409.06289*, 2024.

[15] Saizhuo Wang, Hang Yuan, Lionel M Ni, and Jian Guo, "Quantagent: Seeking holy grail in trading by self-improving large language model," *arXiv preprint arXiv:2402.03755*, 2024.

[16] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang, "Connecting large language models with evolutionary algorithms yields powerful prompt optimizers," in *The Twelfth International Conference on Learning Representations*, 2024.

[17] Fei Liu, Xialiang Tong, Mingxuan Yuan, and Qingfu Zhang, "Algorithm evolution using large language model," 2023.

[18] Robert Tjarko Lange, Yingtao Tian, and Yujin Tang, "Large language models as evolution strategies," 2024.

[19] Fei Liu, Tong Xialiang, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang, "Evolution of heuristics: Towards efficient automatic algorithm design using large language model," in *Forty-first International Conference on Machine Learning*, 2024.

[20] Bernardino Romera-Paredes et al., "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, no. 7995, pp. 468–475, 2024.

[21] Mingkai Zheng, Xiu Su, Shan You, Fei Wang, Chen Qian, Chang Xu, and Samuel Albanie, "Can gpt-4 perform neural architecture search?," *arXiv preprint arXiv:2304.10970*, 2023.

[22] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al., "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24824–24837, 2022.

[23] Noah Hollmann, Samuel Müller, et al., "Large language models for automated data science: Introducing CAAFE for context-aware automated feature engineering," in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[24] Holland John, "Adaptation in natural and artificial systems," 1975.

[25] Zura Kakushadze, "101 formulaic alphas," 2016.

[26] Adam Lipowski and Dorota Lipowska, "Roulette-wheel selection via stochastic acceptance," *Physica A: Statistical Mechanics and its Applications*, vol. 391, no. 6, pp. 2193–2196, 2012.

[27] OpenAI, "Gpt-4 technical report," 2024.

[28] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al., "Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[29] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al., "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[30] Xiao Yang, Weiqing Liu, et al., "Qlib: An ai-oriented quantitative investment platform," 2020.

Authorized licensed use limited to: Indian Institute of Technology - Jodhpur. Downloaded on October 18,2026 at 14:43:35 UTC from IEEE Xplore. Restrictions apply.